Embrace New AI Ways of Working

A comprehensive guide to fundamental changes in how we will work and operate as a Product Engineering organization (Dev, PM, Program & QA) in the World of AI

Author: Jigar Dafda, Chief Technology and Product Officer - Fynd

---

1. Single Repo per Product, Easy to Clone, Setup and Contribute, Easy Vibing Mode, and Homogenous Tech Stack

- For each product, we will consolidate all related microservices into a single Git repo. For example, all services of Boltic will live under one repo. Our long-term goal is for Fynd to operate with only ~25 repositories in total.
- **A unified repo does not mean a single service.** We will continue to follow a microservices architecture.
- This change is purely at the git/code repo level - creating **a unified codebase per product to strategically improve accessibility and discoverability, simplify onboarding, ensure stack consistency, make the codebase easier to navigate, enable cross-service contributions, and encourage broader participation** from both tech and non-tech team members.
- It will allow non-tech team members to contribute meaningfully and make "easy" and "medium" development tasks genuinely accessible. For example, Product Managers should be able to contribute at an SDE-1 level with the help of AI, and the Design team should be able to directly support UI/UX improvements.

2. Repo as the Central Knowledge Hub

- **The repo codebase should become the central hub for all knowledge bases and documentation.** We should create a directory within the same repo, such as */docs*, and store everything there instead of spreading information across Jira tickets, Quip docs, PDFs, and other data.
- Skills, documentation, rules, hooks, and runtime versions should be added centrally for all products and shared across services. Rather than being scattered across multiple platforms, everything should reside within the repo. Even if the content originates as an image, PDF, diagram, or any other format, it should preferably be converted and stored in Markdown to improve AI visibility and contextual understanding.
- The repo will effectively serve as a fully indexed knowledge base for AI-powered IDEs such as Cursor or Claude. These AI IDEs will automatically consume the contextual information stored within the repo and **enable a unified operating surface across all stakeholders**.
- The goal is to ensure that both technical and non-technical stakeholders can operate on the same surface. QA engineers, program managers, and product managers should be able to explore and question the code directly to understand workflows, instead of always relying on developers. This minimizes the loss of information during translation and further assists by helping stakeholders understand the logic and reasoning directly from the codebase.

3. Stop Coding APIs, Start Coding Agents

- Instead of focusing on building traditional APIs, **start building agents**. Shift your mindset toward **designing intelligent, autonomous systems that can reason, decide, and act**.
- Explore frameworks like LangGraph, OpenAI Agents, CrewAI, and similar tools. These ecosystems already provide the foundations you need - so don't spend your time reinventing basic infrastructure.
- CRUD is no longer a differentiator. Today, CRUD is essentially a "hello world" prompt. With AI-powered IDEs like Cursor or Claude, anyone who can write clear English can generate CRUD applications in under an hour.

4. Feature-Based Teams, Not Service-Based Teams

- We will no longer operate as Frontend teams, Backend teams, OMS teams, or any other function-based or service-aligned groups. We will **operate as feature-first, outcome-driven teams**.
- Every team will own a feature end-to-end - from problem definition and design to development, integration, testing, release, and post-launch impact. There will be no fragmented ownership, no layer-based handoffs, and no "that's another team's responsibility."
- When a feature comes in, a single team takes full accountability to ship it - completely and correctly.
- **End-to-end ownership. Clear accountability. Real outcomes.**

5. Go Beyond Role Boundaries

- **Boundaries between Devs, QAs, PMs, and Program are essentially becoming invisible.**
- AI has dramatically lowered the barrier to execution. You no longer need deep specialization to contribute meaningfully across frontend, backend, testing, automation, or documentation. With AI IDEs and plain-English interfaces, anyone can build CRUD apps, generate UI, write scripts, or create tests.
- This means roles like Dev, PM, Program, and QA cannot stay boxed into traditional definitions. PMs and Program Managers should become technically fluent and prototype with AI. Engineers should engage directly with customers and shape solutions, not just implement tickets. The future belongs to those who think in terms of end-to-end problem ownership rather than functional silos.

6. Code Is Cheap, Ownership + Innovation/Deep-Thinking + Speed Matter

- In the AI era, generating code is no longer a scarce skill. What's **scarce is clarity of thought, strong judgment, deep problem understanding, and the ability to move fast with conviction**.
- Ownership, innovation, and speed now define value. The differentiator is not "Can you code?" but "Can you identify the right problem, design the right approach, and drive it to impact quickly?" As AI compresses execution time, the premium shifts to decision-making quality and initiative. Those who wait for defined tasks or hide behind role boundaries will struggle; those who take initiative and think deeply will compound their impact.
- Either expand your role beyond defined boundaries, or become an ultra-expert in solving problems that go far beyond what AI can handle - such as deep performance engineering, advanced security analysis with real-world impact that AI cannot easily detect, large-scale architecture design, and complex domain challenges built on non-public knowledge.

7. QA Must Move Beyond Manual Work

- **You need to start upgrading yourself - from just a quality engineer to a full-on Quality and automation engineer.**
- Especially for QA, it has never been this easy to do automation. Till now, you had to write a lot of code to do any sort of automation. Now it takes barely 15 minutes to create any sort of automation script, and that too with the highest code quality possible.
- You guys have the best edge and this is the time for you guys to shine, considering you guys are in the verification phase of SDLC. Building has become easy, but verification is still unsolved and it needs people who can test the built software through automation as well as manually. You can very much start fixing small bugs yourself. At this point in time, you need to make sure that you upgrade yourself.
- If you are still doing only manual work, it is extremely counterproductive and a waste of both your time and the company's time. It reflects a lack of initiative to think beyond the task at hand

8. Keep Architecture Simple Until You Have Scale, Avoid Premature Complexity for New Projects

- **Do not introduce complex tech stacks before you have real scale.** In the early stages, speed and clarity matter far more than architectural sophistication. Start with something simple, easy to comprehend, and with minimal moving parts so you can build and iterate quickly.
- **Avoid premature complexity in new projects.** Adding Kafka, Redis, or other heavy components for small extensions is often a waste of memory, engineering effort, and operational overhead. Complexity should be earned, not assumed.
- **Optimize your fundamentals first.** For example, if your database indexes are not fully optimized, adding Redis is just masking inefficiency. Build simple, scale when necessary, and introduce advanced components only when the problem genuinely demands them.

9. Don't Fall for the "If I Do It with AI, I Will Forget How to Code" Hoax

- Let's not fall for the misconception that using AI will make us forget how to code.
- That argument is similar to saying we shouldn't use washing machines because we might forget how to wash clothes by hand. What truly matters is the outcome and the value delivered - not the manual effort behind it.
- We no longer write most systems in Assembly or even C, despite their efficiency and speed, because higher-level tools allow us to move faster and build more complex systems efficiently. Technology evolves, and so should we.
- AI is simply the next step in that evolution. It accelerates development, enhances productivity, and allows us to focus on higher-order problem-solving rather than repetitive tasks.
- **At our core, we are problem solvers. Writing code is just one of the tools in our toolkit** - it enables the solution, but it is not the solution itself.

10. Be Curious, Ask AI Everything - No Question is Too Stupid

- Don’t be afraid to ask AI every single question you have, no matter how basic or “stupid” it may seem. There is no judgment - **AI is endlessly patient and available 24/7**.
- **Use AI as your personal teacher.** Let it explain concepts to you step by step until you understand something end-to-end. The goal is deep understanding, not surface-level familiarity.
- The fastest way to fill knowledge gaps is to simply ask. Instead of pretending you know something or spending hours Googling, just ask AI directly - it will break things down for you in seconds
- Learning is a two-way street. As you ask AI questions, you also teach it your context - your codebase, your product, your constraints. The more you interact, the better it gets at helping you.
- The people who will grow the fastest are not the ones who already know everything—they are the ones who are curious enough to keep asking until they truly get it.